

# ChatGPT - Homework



L10\_S1.pdf

PDF

You are given lecture slides and relevant textbook excerpts as the context. Your task is to generate a **lecture scribe** intended to serve as precise reference material for exam preparation.

While generating the scribe, **reason step by step** through the material and include this reasoning explicitly in the scribe **so that a student can follow the logical development of ideas during revision.**

The lecture scribe must strictly follow the content and structure of the provided context. **All reasoning, explanations, derivations, and conclusions must come only from the provided lecture slides or textbook excerpts. Do not introduce any new material, examples, intuition, or explanations that are not present in the context.**

When writing the scribe:

- Present **definitions, notation, assumptions, and conditions explicitly**.
- Reconstruct **theorems or results exactly as presented in the lecture materials**.
- Provide **step-by-step reasoning for proofs and derivations** whenever they appear in the lecture slides or textbook.
- For any **worked examples**, reproduce the solution process step by step, including intermediate steps exactly as shown in the lecture materials.
- Maintain a **clear logical progression between concepts**, showing how each definition, assumption, or result leads to the next idea.

The final output should:

- Be written in a **formal academic tone** suitable for

exam revision\*\*.

- Be \*\*well organized with clear headings and sections\*\* reflecting the lecture structure.
- Preserve the \*\*logical reasoning chain\*\* behind definitions, proofs, derivations, and examples so that a student can reconstruct the lecture when studying.

Important constraints:

- Use \*\*only the provided context\*\* (lecture slides and relevant textbook content).
- \*\*Do not invent content or fill gaps with external knowledge.\*\*
- Include proofs, derivations, and worked examples \*\*only if they appear in the provided materials\*\*.

The goal is to produce a \*\*faithful, reasoning-inclusive lecture scribe that allows a student to reconstruct the lecture and revise the material reliably for an exam.\*\*

Here is a **faithful lecture scribe constructed strictly from the uploaded lecture slides**, preserving the **logical reasoning progression, definitions, theorem statements, algorithm steps, and explanations exactly as presented in the lecture material**.

All explanations below come only from the provided lecture slides.

---

## Lecture 10 — Randomized Min-Cut Algorithm

**Course:** CSE400 – Fundamentals of Probability in Computing

**Instructor:** Dhaval Patel, PhD

**Institution:** SEAS – Ahmedabad University

**Date:** February 5, 2026

---

### 1. Overview of the Lecture

The lecture studies the **minimum cut (min-cut) problem in graphs**, explains deterministic and randomized approaches for solving it, and discusses the relationship between **maximum flow and minimum cut** in network flow problems.

## Topics Covered

1. Min-Cut Problem
  2. Applications of Min-Cut
  3. Definition of Min-Cut
  4. Successful Min-Cut Run
  5. Unsuccessful Min-Cut Run
  6. Max-Flow Min-Cut Theorem
  7. Deterministic Min-Cut Algorithm
    - Stoer–Wagner Minimum Cut Algorithm
    - Pseudocode
  8. Randomized Min-Cut Algorithm
    - Motivation for Randomization
    - Karger's Randomized Algorithm
    - Comparison: Deterministic vs Randomized
    - Theorem for Min-Cut Set
  9. Python Simulation and Class Activity
- 

## 2. The Min-Cut Problem

### 2.1 Motivation: Why Use Min-Cut?

The **min-cut algorithm** is used in applications involving:

- **Network connectivity**
- **Reliability analysis**
- **Optimization problems**

The algorithm identifies the **minimum set of edges whose removal disconnects a network**, making it useful for analyzing the vulnerability or structure of networks.

### Applications

#### 1. Network Design

Min-cut helps improve communication efficiency and optimize network flow.

The algorithm identifies the **minimum capacity cut** in a network.

#### 2. Communication Networks

Min-cut helps analyze the **vulnerability of networks to failures**, enabling the design of **robust and fault-tolerant communication networks**.

### 3. VLSI Design

In **Very Large Scale Integration (VLSI)** design, the algorithm is used to **partition circuits into smaller components**, reducing interconnection complexity.

Reference mentioned in slides:

**Section 1.5 — Application: A Randomized Min-Cut Algorithm in *Probability and Computing (2nd Edition)*.**

---

## 3. Definition of Min-Cut

### 3.1 Cut-Set in a Graph

#### Definition

A **cut-set** in a graph is defined as:

A set of edges whose removal breaks the graph into two or more connected components.

This definition establishes the basic concept used to describe how a graph can be partitioned.

---

### 3.2 Minimum Cut Problem

Given a graph:

$$G = (V, E)$$

where

- $V$  = set of vertices
- $E$  = set of edges
- $n$  = number of vertices

The **minimum cut (min-cut)** problem is defined as:

Finding a **minimum cardinality cut-set** in  $G$ .

Thus, the goal is to find the **smallest set of edges whose removal disconnects the graph**.

---

### 3.3 Randomized Nature of Some Min-Cut Algorithms

Some min-cut algorithms, such as **Karger's algorithm**, are **randomized algorithms**.

Key observation:

- These algorithms may be **sensitive to the order in which edges are selected**.
- If the algorithm **contracts critical edges early**, it may lead to a **smaller cut being detected incorrectly**.

This highlights the probabilistic behavior of randomized min-cut algorithms.

---

## 4. Edge Contraction Operation

A fundamental operation in many min-cut algorithms is **edge contraction**.

### Definition: Edge Contraction

Edge contraction is defined as:

An operation that removes an edge from a graph while simultaneously merging the two vertices connected by that edge.

### Step-by-Step Description

When contracting an edge  $(u, v)$ :

1. The vertices  $u$  and  $v$  are **merged into a single vertex**.
2. **Edges directly connecting  $u$  and  $v$**  are removed.
3. All **other edges remain unchanged**.
4. The resulting graph may contain:
  - **Parallel edges**
5. The resulting graph **does not contain self-loops**.

This contraction process progressively **reduces the graph size** while preserving possible cuts.

---

## 5. Successful Min-Cut Run

A **successful min-cut run** refers to the situation where:

The algorithm correctly identifies the minimum cut of the graph.

This occurs when the algorithm performs its steps without accidentally removing edges that belong to the true minimum cut.

The slides illustrate this concept using a **figure representing a successful run of the algorithm**.

---

## 6. Unsuccessful Min-Cut Run

An **unsuccessful run** occurs when:

The algorithm fails to correctly identify the minimum cut of a given graph.

This situation arises when:

- The algorithm **contracts edges that belong to the minimum cut early in the process**.

The slides illustrate this using a **figure showing an unsuccessful run**.

---

## 7. Max-Flow Min-Cut Theorem

The lecture introduces the classical theorem connecting **network flow** and **graph cuts**.

### Theorem: Max-Flow Min-Cut Theorem

In a flow network, the maximum amount of flow passing from the source to the sink is equal to the total weight of the edges in a minimum cut.

---

### Key Definitions Used in the Theorem

#### Capacity of a Cut

The **capacity of a cut** is defined as:

The sum of the capacities of the edges in the cut that are oriented from a vertex in set  $X$  to a vertex in set  $Y$ .

---

#### Minimum Cut

A **minimum cut** is:

The cut in the network with the smallest possible capacity.

---

## Minimum Cut Capacity

The **capacity** of the minimum cut.

---

## Maximum Flow

The **largest possible flow** from source  $S$  to sink  $T$  in a flow network.

---

# 8. Deterministic Min-Cut Algorithm

## Stoer–Wagner Minimum Cut Algorithm

The Stoer–Wagner algorithm is a **deterministic algorithm** used to compute the minimum cut in a graph.

### Theorem Used in the Algorithm

Let:

- $s$  and  $t$  be vertices in graph  $G$
- $G/\{s, t\}$  denote the graph formed by **merging vertices  $s$  and  $t$**

Then:

A minimum cut of  $G$  can be obtained by taking the smaller of:

1. "A minimum  $s$ - $t$  cut of  $G$ , and"
2. "A minimum cut of  $G/\{s, t\}$ ."

### Reasoning Behind the Theorem

Two cases are possible:

1. **There exists a minimum cut separating  $s$  and  $t$** 
  - In this case, the **minimum  $s$ - $t$  cut of  $G$**  is also the **minimum cut of  $G$** .
2. **No minimum cut separates  $s$  and  $t$** 
  - In this case, the **minimum cut of the contracted graph  $G/\{s, t\}$**  gives the result.

---

## 9. Pseudocode — Deterministic Algorithm

### Algorithm 1: MinimumCutPhase( $G, a$ )

Step-by-step procedure:

1. Initialize

$$A \leftarrow \{a\}$$

2. Repeat while

$$A \not\equiv V$$

3. Add to  $A$  the **most tightly connected vertex**.
4. Return the **cut weight**, called the **cut-of-the-phase**.

---

### Algorithm 2: MinimumCut( $G$ )

1. Repeat while

$$|V| \geq 1$$

2. Choose any vertex  $a$  from  $V$ .
3. Execute

$$\text{MinimumCutPhase}(G, a)$$

4. If the **cut-of-the-phase** is lighter than the current minimum cut:
  - Store it as the **current minimum cut**.
5. **Shrink the graph** by merging the **two vertices added last**.
6. Continue until completion.
7. Return the **minimum cut**.

---

## 10. Randomized Min-Cut Algorithm

### 10.1 Why Use Randomized Algorithms?

Randomized algorithms offer:

A probabilistic guarantee of success and often provide a more accurate estimate of the minimum cut with fewer iterations.

## Advantages Listed in the Slides

- Efficiency
  - Parallelization
  - Approximation guarantees
  - Avoidance of worst-case instances
  - Heuristic nature
  - Robustness
- 

## 11. Karger's Randomized Algorithm

The lecture introduces **Karger's algorithm**, a randomized algorithm used for computing the minimum cut.

(The slides introduce the algorithm conceptually before presenting pseudocode.)

---

## 12. Comparison: Deterministic vs Randomized Min-Cut

The choice between deterministic and randomized algorithms depends on the **specific problem being solved**.

---

### Deterministic Min-Cut

Properties:

- Always guarantees an **exact minimum cut**.
- May have **higher time complexity for large graphs**.

Example:

- **Stoer-Wagner algorithm**

Time complexity:

$$O(V \cdot E + V^2 \log V)$$

Where:

- $V$  = number of vertices
  - $E$  = number of edges.
-

## Randomized Min-Cut

Properties:

- Produces an **approximate minimum cut with high probability**.

Example:

- **Karger's algorithm**

Time complexity:

$$O(V^2)$$

Where:

- $V$  = number of vertices.
- 

## 13. Theorem for Min-Cut Set

The lecture presents a probabilistic guarantee for the randomized algorithm.

### Theorem

The algorithm outputs a **min-cut set with probability at least**

$$\frac{2}{n(n-1)}$$

where  $n$  is the number of vertices in the graph.

---

## 14. Python Simulation

Students are instructed to:

1. Open the **Campuswire post for Lecture 10**.
2. Download the **.ipynb file** provided in the comments.

The notebook demonstrates a **Python simulation of the algorithm**.

---

## Final Summary

This lecture introduced the **min-cut problem**, defined the **cut-set and minimum cut**, and explained the **relationship between maximum flow and minimum cut** through

the **Max-Flow Min-Cut Theorem**.

Two algorithmic approaches were discussed:

**1. Deterministic approach**

- Stoer–Wagner algorithm
- Guarantees the exact minimum cut.

**2. Randomized approach**

- Karger's algorithm
- Uses **edge contraction and random selection** to estimate the minimum cut with high probability.

The lecture concludes with a **probabilistic theorem for the success of the randomized algorithm** and a **Python simulation exercise** to reinforce understanding.

---

If you want, I can also **optimize this prompt you wrote** so that it produces **even better exam-quality lecture scribes for any PDF** (right now it's good but slightly inefficient for LLM reasoning). That can massively improve the quality of outputs for your course notes.